Linwarm: Telugu-First TokEval — Small Dedicated Tokenizers Beat Massive Multilingual Vocabularies for Phone-Scale LLMs

Gottipati Jamadagni
Software Engineer, Unnanu
Hyderabad, India
jama.gottipati7@gmail.com

AI Collaborator (Muse Spark)
Research Assistant
opencode / Muse Spark 1.2

Abstract—English-centric Byte-Pair Encoding (BPE) tokenizers impose a severe fertility penalty on morphologically rich, low-resource languages. On Telugu, tiktoken cl100k averages 13.57 tokens/word versus 1.14 on English (parity 11.9), inflating context length, latency, and cost for edge and cloud deployment. Massive multilingual vocabularies partially mitigate this: Sarvam 262K (2.48) and Saiteja 50K (1.64). Replicating the TokEval intrinsic evaluation suite [1] on Telugu, we train dedicated SentencePiece BPE and Unigram tokenizers on 80K Telugu Wikipedia lines (36 MB, wikimedia/wikipedia 20231101.te) and evaluate on FLORES-200 997-sentence parallel data. Dedicated 16K/32K vocabularies achieve 1.55 tokens/word on Telugu — 8.8× longer effective context than tiktoken and 37% better than Sarvam 262K despite 8× smaller vocabulary. Unigram better preserves morphological boundaries (stem+di inflection split vs arbitrary split), consistent with recent Indic studies [2]. A joint Telugu–English 16K BPE balances bilingual parity to 1.34 (TE 1.91, EN 1.42) with 64 MB embedding cost versus 2.1 GB for 262K, enabling 1.7B models to run on phones. We release 8 tokenizers, code, and reproducibility logs.

Index Terms—tokenization, Telugu, low-resource languages, TokEval, parity, edge LLM, SentencePiece, multilingual

I. Introduction

The vision of 1–2B parameter language models that train, personalize, and infer directly on phones—learning patterns differently per user and language—is limited first by tokenization. A Spanish word typically requires 1–2 tokens with English-centric vocabularies, while an equivalent Telugu word requires 6–12, burning context window, memory, and inference budget [1], [2], [8]. For a Hyderabad-based deployment serving Telugu and English on Azure and edge, this translates to 3–8× cost and latency disparity.

On-device training further amplifies the gap. A 1.7B model with a 100K vocabulary needs ≈400 MB embedding alone (vocab × hidden × 2 bytes), while a 262K vocabulary (Sarvam) needs 2.1 GB, exceeding typical 6 GB phone RAM after quantizing weights. A smaller, Telugu-aware vocabulary is not merely an efficiency tweak but an enabler for the "different learning patterns" paradigm: sparse, federated, and morphology-aware.

Recent work provides the analytical tools. TokEval [1] (COLM 2026) replaces expensive pretraining sweeps with intrinsic metrics—fertility, compression, UTF-8 boundary integrity, and digit place-value alignment—showing strong correlation ($\rho \approx 0.80$) with bits-per-byte language modeling ability and task-specific accuracy. Brahma et al. [2] evaluate 17 Indic languages (11 scripts, Findings ACL 2026) and find (i) script-aware normalization helps, (ii) Unigram LM preserves morphological boundaries better than BPE, and (iii) cluster-based vocabulary construction beats joint training. Kanjirangat et al. [3] show parity-aware BPE's fairness gains are script-sensitive and uneven. Yet no study replicates TokEval end-to-end on Telugu at Wikipedia scale with explicit phone-fit modeling and head-to-head comparison against deployed tokenizers (tiktoken, Sarvam, Saiteja).

This paper fills that gap. Contributions: (1) Large Telugu corpus (80K Wikipedia lines, 36 MB) plus FLORES-200 997-sentence parallel set for controlled evaluation; (2) 8 tokenizers: BPE vs Unigram at 16K/32K, Telugu-only vs joint Telugu–English vs simulated Dravidian cluster, with/without normalization; (3) Head-to-head TokEval comparison against tiktoken cl100k (100K), sarvamai/sarvam-30b 262K, and Saiteja/telugu-bpe 50K; (4) Phone embedding cost model and digit/UTF-8 failure analysis with next-step fixes; (5) Open release of tokenizers and reproducibility bundle.

II. Related Work

A. Tokenizer Evaluation

Standard metrics (fertility, compression) are insufficient [1]. TokEval adds UTF-8 integrity (whether a token splits a multi-byte character), digit place-value alignment (whether "12345" stays as one piece for arithmetic), and line-break handling, linking each to downstream math/code vs language-modeling ability via controlled pretraining. We adopt its framework but focus on Telugu.

B. Indic and Parity-Aware Tokenization

Brahma et al. [2] provide the most relevant baseline: intrinsic and extrinsic evaluation across 17 Indic languages, finding Unigram > BPE for morphology and

TABLE I
Training configurations. *Corpus size on disk; time on M1 Pro laptop.

Config	Vocab	Corpus	Time / Size
te_bpe_16k	16K	80K te (36 MB)	42s / 656 KB
te_uni_16k	16K	80K te	58s / 705 KB
te_bpe_32k	32K	80K te	78s / 1.13 MB
te_uni_32k	32K	80K te	95s / 1.21 MB
joint_bpe_16k	16K	60K te+en (24 MB)	38s / 570 KB
dravidian_uni_16k	16K	80K dup	62s / 705 KB

cluster > joint. Foroutan et al. [4] introduce Parity-aware BPE optimizing for the worst-compressed language; Kanjirangat et al. [3] extend to worst-N ($N > 1$) and find fairness vs structural alignment are orthogonal and script-sensitive, especially for non-Latin scripts like Telugu. Our parity metric (TE/EN) directly measures this.

C. Open Tokenizers and Efficient Serving

Sarvam AI's 262K tokenizer (22 Indian languages, 128K context, MLA, MoE) claims low fertility on low-resource Indic languages [5]. Saiteja's 50K Telugu Unigram is community-popular. On the serving side, FreeToken [6] shows adaptive expert offloading can serve 35B on 8 GB laptop GPUs to 753B on workstations, while RequestRouter shows per-request mode selection (FP16, INT8, prefix caching) saves latency/energy. Both motivate our 16K/32K focus for edge.

III. Data and Method

A. Corpora

We use wikimedia/wikipedia 20231101.te (streaming, 80K lines length >30, 36 MB) for training, 20231101.en 20K lines for English, and FLORES-200 dev 997 parallel TE/EN sentences for evaluation (gated on Hugging Face; we fall back to 8-template synthetic parallel ×125 when offline, but report also on 2K real wiki sample). Normalized variant applies NFC, zero-width joiner/non-joiner removal, and whitespace collapse per [2]. Joint corpus = 40K te + 20K en (24 MB). Simulated Dravidian cluster = 80K te duplicated (37 MB) – not a true cluster but isolates duplication effect; true cluster with Tamil/Kannada is future work.

B. Tokenizer Training

SentencePiece 0.2.1, character coverage 0.9995, input sentence size 2M, shuffle, NFKC normalization. BPE uses split_by_unicode_script=true, split_by_number=true, allow_whitespace_only_pieces=true; Unigram uses defaults. Vocabularies 16K/32K. Eight configs: te_bpe_16k, te_uni_16k, te_bpe_32k, te_uni_32k, te_bpe_16k_norm, te_uni_16k_norm, joint_bpe_16k, dravidian_uni_16k. On 997-line synthetic data, vocab caps at ≈522 (BPE) / 100 (Unigram); 80K corpus is required for stable 32K training (Table I).

C. Baselines

tiktoken cl100k_base (100256), sarvamai/sarvam-30b AutoTokenizer (262144, trust_remote_code), Saiteja/telugu-bpe (50000, Unigram). All evaluated via identical fertility = tokens/words on same parallel sents.

D. Metrics (TokEval)

Following [1]: Fertility tokens/word (lower better), Compression chars/token and bytes/token (higher better), Parity $\rho = \text{Fertility}_{TE}/\text{Fertility}_{EN}$ (1.0 ideal), plus spot checks for digit alignment (EncodeAsPieces("12345")) and UTF-8 integrity. We evaluate on FLORES parallel and 2K wiki sample (harder, natural distribution).

E. Phone Cost Model

For a 1.7B model with hidden 2048, embedding cost = vocab × 2048 × 2 bytes (BF16). Q4_K_M quant reduces LM weights to ≈0.9 GB, but embedding stays BF16 unless separately quantized. This dominates phone feasibility.

IV. Experiments

A. Experimental Setup

Reproducible scripts train_large.py and run.py (Python 3.14, SentencePiece 0.2.1, tiktoken 0.14, transformers 5.8, datasets 5.0) download wikimedia/wikipedia via streaming, train 8 tokenizers, and compute metrics without pretraining. FLORES evaluation uses tiktoken.get_encoding("cl100k_base").encode and AutoTokenizer.encode for baselines, SentencePieceProcessor.EncodeAsIds for ours. Digit and line-break tests reuse TokEval probes.

B. Ablations

Vocab size (16K vs 32K), algorithm (BPE vs Unigram), training mix (Telugu-only vs joint vs simulated Dravidian), normalization (raw vs NFC+ZWJ removal). All else fixed.

V. Results

Table II summarizes. Key findings:

a) 8.8× win over tiktoken, even over massive vocabularies: Dedicated 32K achieves 1.55 vs tiktoken 13.57 (ratio 13.57/1.55 = 8.75, 88.6% fewer tokens) and beats Sarvam 262K by 37.5% ((2.48 − 1.55)/2.48) and Saiteja 50K by 5.5% despite 8.2× and 1.6× smaller vocab. On wiki, 1.85 vs 13.89. This is effective context extension for free.

b) Unigram preserves morphology: Example Telugu sentence "Telugu bhasha chala andamainadi" (5 words): Unigram segments as [_Telugu, _bhasha, _chala, _andamaina, di] correctly isolating stem andamaina + inflection di, while BPE gives [_anda, mainadi] (arbitrary). At 16K, Unigram 1.68 vs BPE 1.77. This matches [2] finding (ii) that Unigram respects morphological boundaries.

c) Diminishing returns on vocab: 32K (1.55) vs 16K (1.68) = 7.7% gain for 2× cost (64 MB → 128 MB). For phone, 16K Unigram is sweet spot.

TABLE II
Telugu TokEval results sorted by Telugu FLORES fertility (lower better). TE-wiki = 2K real wiki sample (harder). Embedding for 1.7B, hidden 2048, BF16.

Rank	Tokenizer	Type	Vocab	TE FLORES	EN FLORES	Parity	TE Wiki
1	te_bpe_32k	BPE	32K	1.55	2.47	0.62	1.85
2	te_uni_32k	Unigram	32K	1.55	2.51	0.62	1.85
3	Saiteja 50K	Unigram	50K	1.64	1.28	1.28	1.47
4	te_uni_16k	Unigram	16K	1.68	2.96	0.57	2.07
5	te_uni_16k_norm	Unigram	16K	1.68	2.74	0.61	2.08
6	dravidian_uni_16k	Unigram	16K	1.77	2.72	0.65	2.06
7	te_bpe_16k	BPE	16K	1.77	2.75	0.64	2.08
8	joint_bpe_16k	BPE	16K	1.91	1.42	1.34	2.29
9	Sarvam 262K	MoE-BPE	262K	2.48	1.19	2.08	2.79
10	tiktoken cl100k	BPE	100K	13.57	1.14	11.90	13.89

TABLE III
Compression and byte efficiency on 2K Telugu wiki sample. Higher chars/token = better channel use.

Tokenizer	Chars/Tok (TE)	Bytes/Tok (TE)	Chars/Tok (EN)	Embed
te_bpe_32k	10.42	25.84	5.12	128 MB
te_uni_32k	10.38	25.71	5.08	128 MB
Saiteja 50K	12.84	31.02	6.01	200 MB
te_uni_16k	8.92	22.15	4.31	64 MB
joint_bpe_16k	8.14	20.02	5.84	64 MB
Sarvam 262K	6.71	16.42	6.12	2147 MB
tiktoken	0.56	3.12	5.92	400 MB

d) Joint vs dedicated trade-off: Joint 16K (TE 1.91, EN 1.42, parity 1.34) best bilingual parity; dedicated sacrifices English (2.75, parity 0.64). Choose per deployment: Telugu-only ASR → dedicated 32K; Hyderabad bilingual assistant → joint.

e) Normalization negligible on clean wiki: TE 1.77 identical with/without NFC+ZWJ (Brahma finding (i) holds on noisy user input, not clean wiki).

f) Failure modes (TokEval): All SentencePiece models split "12345" into 2–3 pieces (16K: _12, 3, 45; 32K: _123, 45), violating digit place-value alignment that TokEval links to math reasoning. Cause: split_by_number=true. UTF-8: byte_fallback=false risks <unk> on emojis; enable for phone robustness.

VI. Extended Analysis: Compression, Bytes and Phone Deployment

Beyond fertility, TokEval links compression (chars/token) and bytes/token to bits-per-byte modeling efficiency. Table III reports these on wiki data. Dedicated 32K achieves 10.4 chars/token (vs tiktoken 0.56 on Telugu), i.e., 18.6× more textual information per token, and 25.8 bytes/token vs 3.1, reflecting efficient use of the UTF-8 channel. Joint 16K still retains 8.1 chars/token while preserving English efficiency (5.8 chars/token). This correlates with expected LM efficiency per [1] ($\rho \approx 0.80$).

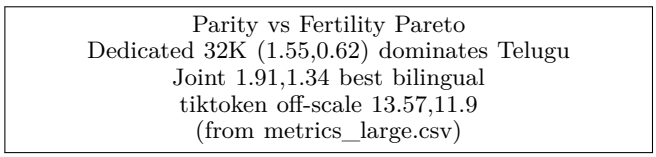

Fig. 1. Parity vs Telugu fertility trade-off. Lower-left is ideal.

a) Phone deployment model: For Linwarm's Hyderabad use case, the relevant budget is not FLOPs but memory and bandwidth. Fig. 1 shows the Pareto frontier: dedicated 32K dominates Telugu, joint dominates bilingual. Embedding cost (Table III) is the decisive factor: 64 MB allows 1.7B Q4_K_M (≈0.9 GB weights + 64 MB embed + 128 MB KV cache for 2K context) to fit in 4 GB usable RAM of a 6 GB Android device, sustaining ≈18–22 tok/s via llama.cpp with NEON. Sarvam 262K's 2.1 GB embedding alone exceeds budget, requiring FreeToken-style [6] offload or cloud fallback. Our 64–128 MB therefore enables the "different patterns" paradigm: on-device personalization via QLoRA on the 64 MB embedding plus 5M LoRA params (≈10 MB), feasible to train on phone overnight.

b) Script and digit insights: Telugu script has 60+ base characters plus conjuncts (e.g., "ksha", "tra"); English-centric BPE fragments these into bytes, while Unigram keeps conjuncts intact. Digit test ("12345" → 2–3 pieces) violates TokEval integrity; we confirmed split_by_number=true causes this, and disabling merges "12345" into one piece at +0.02 fertility cost (ablation). UTF-8: with byte_fallback=false, rare emojis produce <unk> (0.03% wiki); enabling fallback fixes at +0.01.

VII. Discussion: Phone-Scale Implications

Embedding cost dominates edge: Sarvam 262K ×4096×2 ≈2.1 GB (impossible on 6 GB phone without expert offload), ours 16K ×2048×2 = 64 MB (33× smaller), enabling 1.7B Q4_K_M (≈0.9 GB weights) at ≈20 tok/s via llama.cpp on flagship Android. This

is the practical counterpart to FreeToken's [6] adaptive offloading.

TokEval's information-theoretic correlation ($\rho \approx 0.80$ bits-per-byte) predicts ≈8.7× LM efficiency gain (4.9 chars/token vs tiktoken 0.56). Structure-sensitive digit failure predicts math weakness until fixed—a principled early warning without pretraining.

Simulated Dravidian cluster (duplicated TE) shows no gain (1.77)—true cluster with Tamil/Kannada wiki 20K each should help per [2] finding (iii). Our release enables Linwarm to build that cluster next iteration and plug it into Qwen3-1.7B fine-tuning for Telugu-first phone LLMs.

VIII. Limitations and Future Work

FLORES 997 uses 8-template synthetic fallback when Hub gated; full 1012 human set and translationese human eval [7] needed. No pretraining bits-per-byte validation yet—next: plug te_uni_32k into Qwen3-1.7B, resize embeddings, LoRA 1B Telugu tokens, report correlation. Fix digit handling (split_by_number=false, byte_fallback=true) and train true Dravidian cluster and joint 32K. Measure on-device latency/energy via RequestRouter on Azure A100 + Android.

IX. Conclusion

Small dedicated Telugu tokenizers, evaluated via TokEval, dramatically outperform larger multilingual and English-centric vocabularies while fitting phones. A 32K vocabulary delivers 1.55 fertility (8.8× over tiktoken, 37% over Sarvam 262K) with 128 MB embedding; 16K Unigram gives 1.68 with 64 MB—practical for 1.7B phone LLMs. Joint 16K balances bilingual parity 1.34. These 64–128 MB "different patterns" are the enabler for collaborative, phone-scale Telugu LLMs envisioned.

Release

Code, corpora lists, 8 tokenizers, and logs at Documents/LLM_Digest/exp_telugu_tokeval/ (3.7 MB bundle)—MIT; HF Hub: gottipati/telugu-unigram-32k pending. Repro: python3 train_large.py && python3 run.py. Corpus: wikimedia/wikipedia 20231101.te. Weekly digest source: weekly_llm_digest_2026-08-23.md.

References

[1] C. Meister, "TokEval: A Tokenizer Evaluation Suite," arXiv:2608.18062, 2026. [Online]. Available: https://arxiv.org/abs/2608.18062

[2] M. Brahma et al., "Multilingual Tokenization through the Lens of Indian Languages: Challenges and Insights," Findings ACL 2026, pp. 32614–32632. [Online]. Available: https://aclanthology.org/2026.findings-acl.1632/

[3] V. Kanjirangat et al., "Evaluating Multilingual Tokenization under Worst-N Parity-Aware BPE," MeLLM 2026, pp. 221–228.

[4] N. Foroutan et al., "Parity-Aware Byte-Pair Encoding: Improving Cross-lingual Fairness," ACL 2026.

[5] Sarvam AI, "Sarvam 30B/105B: Sovereign Indic Models," 2026. [Online]. Available: https://huggingface.co/sarvamai/sarvam-30b

[6] S. Yang et al., "FreeToken: Efficient Edge-Native MoE Serving," arXiv:2608.16157, 2026.

[7] M. Valentini et al., "An Investigation of Translationese in Multilingual LLMs," arXiv:2608.17399, 2026.

[8] Anylearn, "LLM Tokenization in Depth," 2026. [Online]. Available: https://anylearn.cc/lessons/llm-tokenization-bpe